LOCAL AI APP DESIGN & OPTIMIZATION BLUEPRINT

High-Performance Offline Architectures for React Native & Expo Hackathon Apps

1. Optimized Local Architecture Setup

To build a high-performance offline document analysis app using React Native and Expo without traditional server dependencies, you must swap out latency-heavy standard bindings with native plugins using the Expo Modules API.

• **Local Inference Engine**: Leverage **React Native ExecuTorch** or optimized llama.rn bindings. This allows models like Qwen3 0.6B to interface directly with on-device hardware acceleration backends (Vulkan on Android, Metal/CoreML on iOS) bypassing CPU single-threaded execution bottlenecks.

• **On-Device Document Extractor**: Implement **expo-ocr-kit**. It calls Google ML Kit and Apple Vision frameworks natively. Downsample heavy camera payloads or multi-page documents in raw device memory prior to text recognition to prevent Out-Of-Memory (OOM) fatal crashes.

2. Custom Configuration: metro.config.js

By default, Metro ignores large model binary extensions (like .gguf, .pt, or .bin). You must configure your bundler to asset-pack these local assets directly inside your application package binary without internet dependencies:

```
const { getDefaultConfig } = require('expo/metro-config');
const config = getDefaultConfig(__dirname);

// Extend assetExtensions to include local model files
config.resolver.assetExts.push('gguf', 'pt', 'bin', 'json');

module.exports = config;
```

3. System Prompt Recipe & Anchor Control

Small models (sub-1B parameters) degrade rapidly into hallucinations if given vague constraints. Enforce rigid boundaries using an anchored system prompt recipe:

```
[SYSTEM PROMPT RECIPE]
You are a highly restrictive legal document assistant. Your task is to extract
information based solely on the provided CONTEXT text blocks. Follow these
absolute constraints:
1. Rely exclusively on explicit facts found within the context. Do not deduce or
infer.
2. If a query asks about information not explicitly contained inside the provided
text, reply exactly with: "Information not found in document."
3. Strictly refuse to generate speculative details, legal assessments, or
assumptions.

[CONTEXT]
```

```
{extracted_ocr_text_blocks}

[USER QUERY]
{user_skim_question}
```

4. Anti-Hallucination Schemas via GBNF Grammars

To completely eliminate conversational fluff and guarantee deterministic structured outputs, clamp the text generation engine with a GBNF Grammar rule mapping strictly to valid structural JSON schema objects:

```
root ::= object
object ::= "{" space string space ":" space value space "}"
value ::= string | number
string ::= '"' [^"\\]* '"'
number ::= [0-9]+
space ::= [ \t\n\r]*
```

5. Local UI Quality Control Logic

Implement a quick programmatic verification check to compute text block viability prior to passing data payloads down to the model context window. This guards the model against processing distorted images or faulty camera scans:

```
function verifyScanQuality(ocrBlocks) {
const unreadablePattern = /[^a-zA-Z0-9\s,.:()\-\/%]/g;
let garbageCount = 0;
let totalChars = 0;

ocrBlocks.forEach(block => {
totalChars += block.text.length;
const matches = block.text.match(unreadablePattern);
if (matches) garbageCount += matches.length;
});

const garbageRatio = totalChars > 0 ? (garbageCount / totalChars) : 1;
return garbageRatio < 0.25; // Rejects if > 25% symbols/gibberish
}
```

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.